

GPU-Accelerated Physics-Based Urban Flood Modelling

A production comparison of CuPy/CUDA and JAX/XLA

Sanjeev Kumawat

M.Sc. Student, Indian Institute of Technology Hyderabad

Under the guidance of **Udit Bhatia**

Associate Professor, Indian Institute of Gandhinagar



భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్
भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

1. Research question and completed contribution

Central question

Can two GPU programming systems execute the same city-scale physics-based flood model and remain numerically equivalent under predefined scientific criteria?

The completed study implements one production model through **CuPy/CUDA** and **JAX/XLA**. Both paths use the same mesh, terrain, resistance, rainfall, infiltration, drainage, recharge, boundary forcing, precision and output schedule.

Completed evidence

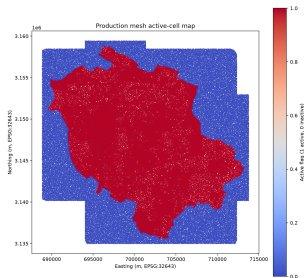
- ▶ Full 24-hour production runs on one GPU
- ▶ Cross-backend parity and repeatability
- ▶ Mass-ledger and software verification
- ▶ Repeated secondary GPU benchmark

Source: HF_CUPY_JAX_COMPLETED_PROJECT.

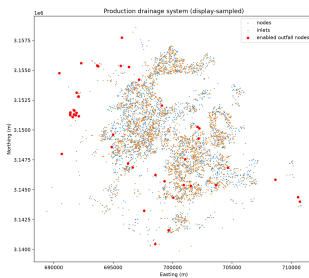
Scientific interpretation

The evidence establishes implementation correctness and numerical consistency. It does not convert model output into observed flood truth; physical accuracy remains a separate validation question.

2. Production domain and drainage network



Active and inactive surface triangles



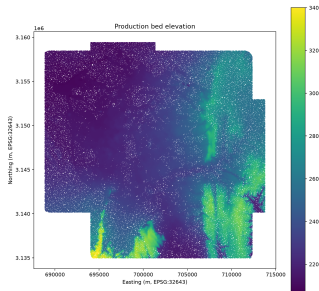
Surface-associated drainage network

Mesh vertices	499,901
Triangles	981,880
Active triangles	817,573
Surface edges	1,490,015
Active area	297.966 km ²
Exterior edges	34,390
Drainage nodes	118,768
Drainage links	139,798
Mapped inlets	61,317
Enabled outfalls	46
Recharge records	447

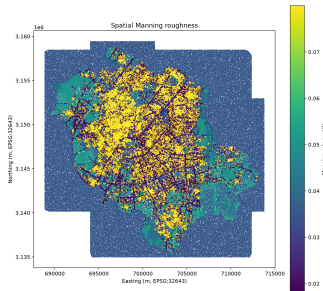
Projected coordinate system:
EPSG:32643.

Source: HF_CUPY_JAX_COMPLETED_PROJECT.

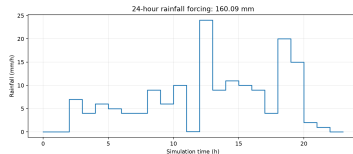
3. Terrain, resistance and rainfall inputs



Bed elevation for all 981,880 cells



Spatial Manning n : 0.018–0.080

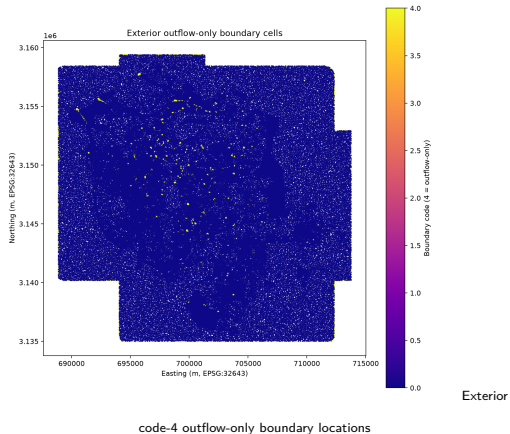


24-hour rainfall: 160.09 mm

Terrain enters hydrostatic reconstruction and hydraulic head. Manning resistance is active friction, not metadata. Rainfall is hourly piecewise constant and spatially uniform, while Horton losses are spatially heterogeneous: $f_0 = 0.0128\text{--}7.2 \text{ mm h}^{-1}$, $f_c = 0.004\text{--}2.25 \text{ mm h}^{-1}$ and $k = 4 \text{ h}^{-1}$.

Source: HF_CUPY_JAX_COMPLETED_PROJECT.

4. Coupled surface, drainage and boundary processes



Surface-node exchange

$$Q_e = C_d A_e \sqrt{2g |H_s - H_n|} \operatorname{sgn}(H_s - H_n)$$

Water can enter mapped inlets, move through capacity-limited links, discharge through outfalls, or return to the surface as surcharge. Recharge captures eligible surface water and tracks storage and removal.

Inactive neighbours behave as reflective walls. Exterior code-4 edges allow outward flux; the prescribed hydrograph is applied separately to 150 receiver cells. Its recorded peak total flow is $95.1714 \text{ m}^3 \text{ s}^{-1}$.

Source: HF_CUPY_JAX_COMPLETED_PROJECT.

5. Governing physics and source terms

The conserved surface state is $\mathbf{U} = [h, hu, hv]^T$ and obeys

$$\frac{\partial \mathbf{U}}{\partial t} + \frac{\partial \mathbf{F}}{\partial x} + \frac{\partial \mathbf{G}}{\partial y} = \mathbf{S}, \quad \mathbf{F} = \begin{bmatrix} hu \\ hu^2 + \frac{1}{2}gh^2 \\ huv \end{bmatrix}, \quad \mathbf{G} = \begin{bmatrix} hv \\ huv \\ hv^2 + \frac{1}{2}gh^2 \end{bmatrix}.$$

Horton infiltration

$$f(\tau) = f_c + (f_0 - f_c)e^{-k\tau}$$

Actual removal is limited by capacity and available surface water.

Semi-implicit Manning friction

$$\mathbf{q}^{n+1} = \frac{\mathbf{q}^*}{1 + \Delta t gn^2 \|\mathbf{u}^*\| / \max(h, h_{\min})^{4/3}}$$

Here $\mathbf{q} = (hu, hv)$ and $h_{\min} = 10^{-5}$ m.

6. Finite-volume method and stable update

For triangular cell i with area A_i ,

$$\mathbf{U}_i^{n+1} = \mathbf{U}_i^n - \frac{\Delta t}{A_i} \sum_{e \in \partial i} L_e \hat{\mathbf{F}}_{e,i} + \Delta t \mathbf{S}_i.$$

The HLL interface flux uses left/right signal speeds s_L, s_R :

$$\hat{\mathbf{F}}_{\text{HLL}} = \frac{s_R \mathbf{F}_L - s_L \mathbf{F}_R + s_L s_R (\mathbf{U}_R - \mathbf{U}_L)}{s_R - s_L}.$$

Well-balanced terrain treatment

$$\begin{aligned} z^* &= \max(z_L, z_R), \\ h_L^* &= \max(h_L + z_L - z^*, 0), \\ h_R^* &= \max(h_R + z_R - z^*, 0). \end{aligned}$$

Adaptive timestep

$$\Delta t = \min \left[1 \text{ s}, 0.85 \min_i \frac{\ell_i}{\|\mathbf{u}_i\| + \sqrt{gh_i}} \right].$$

Flux, friction, rainfall, infiltration, boundary input, network exchange, outfall, surcharge and recharge are applied in the same order in both backends.

7. Two GPU implementations of one model

CuPy/CUDA

Device-resident NumPy-style arrays with fused custom CUDA kernels for surface and drainage operations. Recorded implementation:
`gurugram_flood_backend_fused_cuda`.

JAX/XLA

Immutable array state, scatter reductions and an XLA-compiled `while_loop`. Recorded implementation:
`hybrid_flood_v2_jax_v1_parity`.

	CuPy	JAX
Execution device	CUDA device 0	<code>cuda:0</code>
Production steps	321,645	321,640
Surface precision	<code>float32</code>	<code>float32</code>
GPU fallback	<code>none</code>	<code>none</code>

Platform: NVIDIA RTX PRO 4000 Blackwell, 24,467 MiB; driver 595.84. PyTorch CUDA availability was recorded, but PyTorch did not execute the reported production experiment.

Source: `HF_CUPY_JAX_COMPLETED_PROJECT`.

8. PyTorch/CUDA: capability and project role

What PyTorch provides

An imperative tensor-programming interface, CUDA device execution, automatic differentiation and a mature ecosystem for scientific machine learning and neural-network development.

Its eager execution model is convenient for inspecting intermediate tensors, while graph capture and compilation facilities can optimize suitable workloads.

Source: HF_CUPY_JAX_COMPLETED_PROJECT.

Role in this completed experiment

PyTorch 2.13.0+cu130 was installed and CUDA availability was confirmed on the production system. It did **not** execute the reported flood simulations.

The measured backends were CuPy/CUDA and JAX/XLA. Consequently:

- ▶ no PyTorch runtime is reported;
- ▶ no PyTorch speed ranking is claimed;
- ▶ PyTorch is discussed only as relevant GPU-computing context.

9. JAX: functional programs compiled through XLA

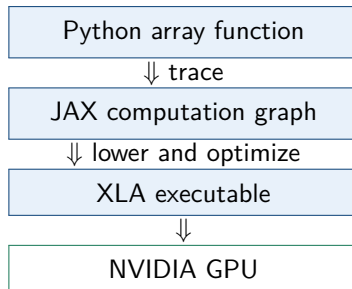
Pure state transition

$$(\mathbf{U}^{n+1}, \mathbf{D}^{n+1}) = \Phi(\mathbf{U}^n, \mathbf{D}^n; \Delta t, \mathcal{G}, \mathcal{P}),$$

where surface state $\mathbf{U}$, drainage state $\mathbf{D}$, geometry $\mathcal{G}$ and physical parameters $\mathcal{P}$ are explicit inputs and outputs.

JAX traces array operations with known shapes and dtypes, constructs an intermediate graph, and lowers it through XLA. XLA can fuse compatible operations and generate device code while JAX preserves an immutable programming interface.

Background synthesis: S. Borah, *High-Performance Scientific Computing in Python: The JAX Ecosystem* (2026); project implementation evidence.



10. JAX/XLA choices in the flood solver

JAX mechanism	Use in this project	Scientific implication
<code>jit</code> and XLA	Compiles the production update	Compilation is distinguished from steady execution
Immutable state	Named state tuples are returned by each update	State evolution is explicit and traceable
Scatter reductions	Accumulates edge and network contributions	Matches unstructured-mesh connectivity
<code>lax.while_loop</code>	Keeps adaptive time integration on device	Avoids a Python-controlled inner timestep loop
Device synchronization	Waits for completed accelerator work	Prevents asynchronous timing bias

Scope actually exercised

The experiment used one GPU and float32 arrays. Automatic differentiation, `vmap`, `pmap`, multi-GPU execution and AI training were not part of the reported production comparison; no benefit from those features is claimed.

Background synthesis: S. Borah, *High-Performance Scientific Computing in Python: The JAX Ecosystem* (2026); project implementation evidence.

11. Verification and experimental protocol

24-hour production verification

Same hashed inputs, float32 state, CFL 0.85, 1 s maximum timestep, 5 m s⁻¹ velocity ceiling and 25 stored states. CuPy is the implementation reference; it is not observational truth.

Eight gates: depth RMSE, flood CSI, volume error, finite/non-negative depth, mass-residual regression, maximum depth, time alignment and source-sink ledger agreement.

Source: HF_CUPY_JAX_COMPLETED_PROJECT.

Completed software baseline

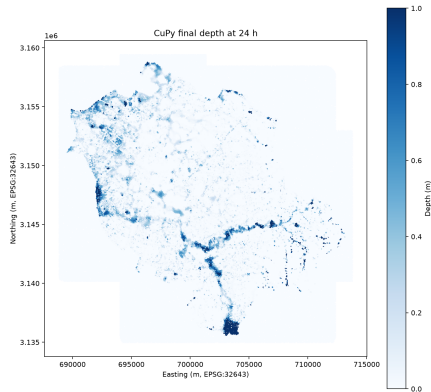
111 passed

37 explicit skips; 0 failures; 0 errors

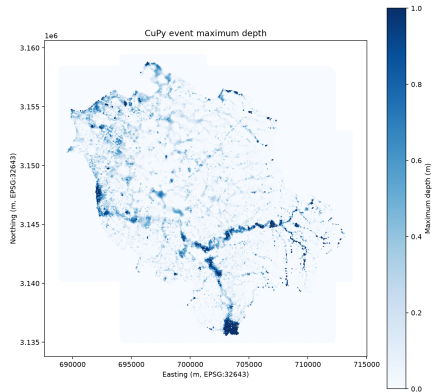
Secondary performance protocol

Identical 60-minute production workload; three warm-ups and ten synchronized measurements per backend. Every timed output passed correctness checks.

12. Production inundation results



surface depth after 24 hours

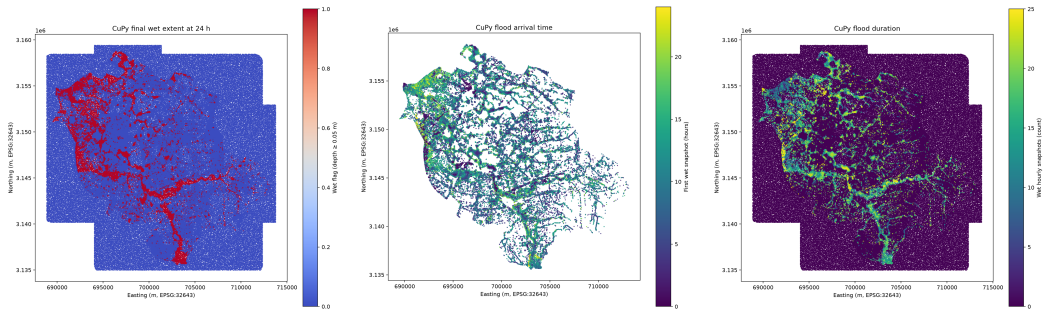


maximum surface depth

CuPy: maximum depth 9.414836884 m; 129,187 final wet triangles. **JAX:** maximum depth 9.414829254 m; 129,180 final wet triangles.

Source: HF_CUPY_JAX_COMPLETED_PROJECT.

13. Flood extent, arrival and duration



Wet extent through the event

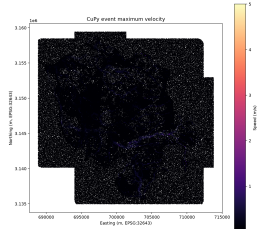
Arrival time at $h > 0.05$ m

Number of wet stored states

Flood extent uses the registered 0.05 m threshold. Arrival and duration are derived from 25 stored states, so their effective temporal resolution is hourly. The duration product is a wet-snapshot count, not a continuously integrated exposure duration.

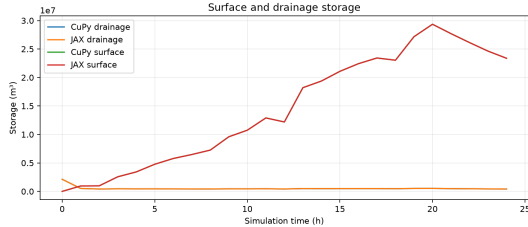
Source: HF_CUPY_JAX_COMPLETED_PROJECT.

14. Velocity, storage and coupled mass response



CuPy event maximum

velocity



drainage-node storage

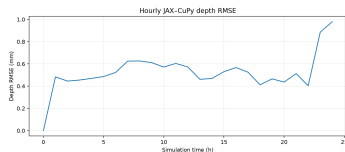
Surface and

Maximum recorded speed was $1.712713 \text{ m s}^{-1}$ for CuPy and $1.712712 \text{ m s}^{-1}$ for JAX, below the 5 m s^{-1} safety ceiling. Final surface storage was 23.378 and 23.376 million m^3 ; final drainage-node storage was 0.4204 million m^3 in both paths.

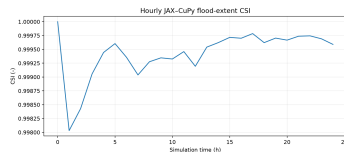
$$R_M = V_{s,0} + V_{n,0} + \sum V_{\text{in}} - \sum V_{\text{loss}} - V_{s,f} - V_{n,f} - V_{r,f}.$$

Source: HF_CUPY_JAX_COMPLETED_PROJECT.

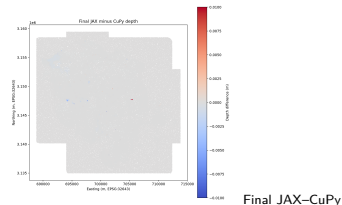
15. Cross-backend numerical parity



Hourly depth RMSE



Hourly flood-extent CSI



Final JAX-CuPy

depth difference

Gate	Limit	Observed	Result
Maximum hourly depth RMSE	≤ 0.05 m	0.000979348 m	Pass
Minimum hourly flood CSI	≥ 0.95	0.998031	Pass
Maximum hourly volume error	$\leq 2\%$	0.033294%	Pass
Snapshot-time mismatch	≤ 1 s	0.318359 s	Pass
Source-sink ledger difference	$\leq 2\%$	1.543653%	Pass

All eight registered parity gates passed.

Source: HF_CUPY_JAX_COMPLETED_PROJECT.

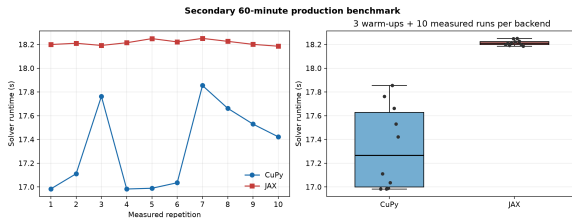
16. Repeatability and measured GPU performance

Independent 24-hour repeat runs

	CuPy	JAX
Depth RMSE (m)	0.000416766	0.000406100
Minimum CSI	0.998740	0.998031
Maximum time offset (s)	0.354492	0.337891
Step-count difference	28	24
Bitwise identical	No	No

Both backends are **numerically repeatable within the approved criteria**, but neither is bitwise deterministic.

Source: HF_CUPY_JAX_COMPLETED_PROJECT.



Three warm-ups and ten measurements per backend

CuPy median: **17.267 s** JAX median: **18.213 s**
JAX was **5.476% slower** for this specific completed 60-minute workload. All timing correctness checks passed.

17. Conclusions and defensible project claims

Primary conclusion

Equivalent CuPy/CUDA and JAX/XLA implementations successfully executed the complete production urban flood model on one NVIDIA GPU, and all registered cross-backend numerical gates passed.

What was demonstrated

- ▶ City-scale coupled surface–drainage execution
- ▶ Numerical parity throughout 24 hours
- ▶ Within-backend numerical repeatability
- ▶ Valid repeated secondary performance comparison

Correct scientific boundary

The results demonstrate GPU implementation consistency, not agreement with observed flood measurements. ANUGA is external reference context only and contributes no production result. The incomplete repeated 24-hour timing series is not used as an official benchmark ratio.

Completed outcome: a verified, reproducible GPU numerical study.

Source: HF_CUPY_JAX_COMPLETED_PROJECT.

<https://github.com/arsanjevk/gpu-urban-flood-cupy-jax>